NAME

ovn-sbctl – Open Virtual Network southbound db management utility

SYNOPSIS

ovn−sbctl [*options*] *command* [*arg*...]

DESCRIPTION

The **ovn−sbctl** program configures the **OVN_Southbound** database by providing a high-level interface to its configuration database. See **ovn−sb**(5) for comprehensive documentation of the database schema.

ovn−sbctl connects to an **ovsdb−server** process that maintains an OVN_Southbound configuration database. Using this connection, it queries and possibly applies changes to the database, depending on the supplied commands.

ovn−sbctl can perform any number of commands in a single run, implemented as a single atomic transaction against the database.

The **ovn−sbctl** command line begins with global options (see **OPTIONS** below for details). The global options are followed by one or more commands. Each command should begin with **−−** by itself as a command-line argument, to separate it from the following commands. (The **−−** before the first command is optional.) The command itself starts with command-specific options, if any, followed by the command name and any arguments.

DAEMON MODE

When it is invoked in the most ordinary way, **ovn−sbctl** connects to an OVSDB server that hosts the southbound database, retrieves a partial copy of the database that is complete enough to do its work, sends a transaction request to the server, and receives and processes the server's reply. In common interactive use, this is fine, but if the database is large, the step in which **ovn−sbctl** retrieves a partial copy of the database can take a long time, which yields poor performance overall.

To improve performance in such a case, **ovn−sbctl** offers a "daemon mode," in which the user first starts **ovn−sbctl** running in the background and afterward uses the daemon to execute operations. Over several **ovn−sbctl** command invocations, this performs better overall because it retrieves a copy of the database only once at the beginning, not once per program run.

Use the **−−detach** option to start an **ovn−sbctl** daemon. With this option, **ovn−sbctl** prints the name of a control socket to stdout. The client should save this name in environment variable **OVN_SB_DAEMON**. Under the Bourne shell this might be done like this:

```
export OVN_SB_DAEMON=$(ovn-sbctl --pidfile --detach)
```

When **OVN_SB_DAEMON** is set, **ovn−sbctl** automatically and transparently uses the daemon to execute its commands.

When the daemon is no longer needed, kill it and unset the environment variable, e.g.:

```
kill $(cat $OVN_RUNDIR/ovn-sbctl.pid)
unset OVN_SB_DAEMON
```

When using daemon mode, an alternative to the **OVN_SB_DAEMON** environment variable is to specify a path for the Unix socket. When starting the ovn-sbctl daemon, specify the **−u** option with a full path to the location of the socket file. Here is an example:

```
ovn-sbctl --detach -u /tmp/mysock.ctl
```

Then to connect to the running daemon, use the **−u** option with the full path to the socket created when the daemon was started:

```
ovn-sbctl -u /tmp/mysock.ctl show
```

When running in daemon mode, **ovn−sbctl** uses the **sbctl_probe_interval** value from the **options** column

in the **SB_Global** table to configure the inactivity probe interval (in milliseconds) for the database connection. The default is 120000 ms (120 seconds). To override, set the value with:

```
ovn-sbctl set SB_Global . options:sbctl_probe_interval=60000
```

Daemon Commands

Daemon mode is internally implemented using the same mechanism used by **ovn-appctl**. One may also use **ovn-appctl** directly with the following commands:

run [*options*] *command* [*arg*...] [**--** [*options*] *command* [*arg*...] ...]
: Instructs the daemon process to run one or more **ovn-sbctl** commands described above and reply with the results of running these commands. Accepts the **--timeout**, **--dry-run**, **--oneline**, and the options described under **Table Formatting Options** in addition to the command-specific options.

exit
: Causes **ovn-sbctl** to gracefully terminate.

OPTIONS

The options listed below affect the behavior of **ovn-sbctl** as a whole. Some individual commands also accept their own options, which are given just before the command name. If the first command on the command line has options, then those options must be separated from the global options by **--**.

ovn-sbctl also accepts options from the **OVN_SBCTL_OPTIONS** environment variable, in the same format as on the command line. Options from the command line override those in the environment.

--db *database*
: The OVSDB database remote to contact. If the **OVN_SB_DB** environment variable is set, its value is used as the default. Otherwise, the default is **unix:/ovnsb_db.sock**, but this default is unlikely to be useful outside of single-machine OVN test environments.

--leader-only
--no-leader-only
: By default, or with **--leader-only**, when the database server is a clustered database, **ovn-sbctl** will avoid servers other than the cluster leader. This ensures that any data that **ovn-sbctl** reads and reports is up-to-date. With **--no-leader-only**, **ovn-sbctl** will use any server in the cluster, which means that for read-only transactions it can report and act on stale data (transactions that modify the database are always serialized even with **--no-leader-only**). Refer to **Understanding Cluster Consistency** in **ovsdb**(7) for more information.

--shuffle-remotes
--no-shuffle-remotes
: By default, or with **--shuffle-remotes**, when there are multiple remotes specified in the OVSDB connection string specified by **--db** or the **OVN_SB_DB** environment variable, the order of the remotes will be shuffled before the client tries to connect. The remotes will be shuffled only once to a new order before the first connection attempt. The following retries, if any, will follow the same new order. The default behavior is to make sure clients of a clustered database can distribute evenly to all members of the cluster. With **--no-shuffle-remotes**, **ovn-sbctl** will use the original order specified in the connection string to connect. This allows a user to specify the preferred order, which is particularly useful for testing.

--no-syslog
: By default, **ovn-sbctl** logs its arguments and the details of any changes that it makes to the system log. This option disables this logging.

  This option is equivalent to **--verbose=sbctl:syslog:warn**.

−−oneline
Modifies the output format so that the output for each command is printed on a single line. New-line characters that would otherwise separate lines are printed as **\n**, and any instances of \ that would otherwise appear in the output are doubled. Prints a blank line for each command that has no output. This option does not affect the formatting of output from the **list** or **find** commands; see **Table Formatting Options** below.

−−dry−run
Prevents **ovn−sbctl** from actually modifying the database.

−t *secs*
−−timeout=*secs*
By default, or with a *secs* of **0**, **ovn−sbctl** waits forever for a response from the database. This option limits runtime to approximately *secs* seconds. If the timeout expires, **ovn−sbctl** will exit with a **SIGALRM** signal. (A timeout would normally happen only if the database cannot be contacted, or if the system is overloaded.)

Daemon Options

−−pidfile[**=***pidfile*]
Causes a file (by default, *program***.pid**) to be created indicating the PID of the running process. If the *pidfile* argument is not specified, or if it does not begin with **/**, then it is created in .

If **−−pidfile** is not specified, no pidfile is created.

−−overwrite−pidfile
By default, when **−−pidfile** is specified and the specified pidfile already exists and is locked by a running process, the daemon refuses to start. Specify **−−overwrite−pidfile** to cause it to instead overwrite the pidfile.

When **−−pidfile** is not specified, this option has no effect.

−−detach
Runs this program as a background process. The process forks, and in the child it starts a new session, closes the standard file descriptors (which has the side effect of disabling logging to the console), and changes its current directory to the root (unless **−−no−chdir** is specified). After the child completes its initialization, the parent exits.

−−monitor
Creates an additional process to monitor this program. If it dies due to a signal that indicates a programming error (**SIGABRT**, **SIGALRM**, **SIGBUS**, **SIGFPE**, **SIGILL**, **SIGPIPE**, **SIGSEGV**, **SIGXCPU**, or **SIGXFSZ**) then the monitor process starts a new copy of it. If the daemon dies or exits for another reason, the monitor process exits.

This option is normally used with **−−detach**, but it also functions without it.

−−no−chdir
By default, when **−−detach** is specified, the daemon changes its current working directory to the root directory after it detaches. Otherwise, invoking the daemon from a carelessly chosen directory would prevent the administrator from unmounting the file system that holds that directory.

Specifying **−−no−chdir** suppresses this behavior, preventing the daemon from changing its current working directory. This may be useful for collecting core files, since it is common behavior to write core dumps into the current working directory and the root directory is not a good directory to use.

This option has no effect when **−−detach** is not specified.

−−no−self−confinement
By default this daemon will try to self-confine itself to work with files under well-known directories determined at build time. It is better to stick with this default behavior and not to use this flag unless some other Access Control is used to confine daemon. Note that in contrast to other access control implementations that are typically enforced from kernel-space (e.g. DAC or MAC), self-

confinement is imposed from the user-space daemon itself and hence should not be considered as a full confinement strategy, but instead should be viewed as an additional layer of security.

−−user=*user***:***group*
Causes this program to run as a different user specified in *user***:***group*, thus dropping most of the root privileges. Short forms *user* and **:***group* are also allowed, with current user or group assumed, respectively. Only daemons started by the root user accepts this argument.

On Linux, daemons will be granted **CAP_IPC_LOCK** and **CAP_NET_BIND_SERVICES** before dropping root privileges. Daemons that interact with a datapath, such as **ovs−vswitchd**, will be granted three additional capabilities, namely **CAP_NET_ADMIN**, **CAP_NET_BROADCAST** and **CAP_NET_RAW**. The capability change will apply even if the new user is root.

Logging Options

−v[*spec*]
−−verbose=[*spec*]
Sets logging levels. Without any *spec*, sets the log level for every module and destination to **dbg**. Otherwise, *spec* is a list of words separated by spaces or commas or colons, up to one from each category below:

- A valid module name, as displayed by the **vlog/list** command on **ovs−appctl**(8), limits the log level change to the specified module.
- **syslog**, **console**, or **file**, to limit the log level change to only to the system log, to the console, or to a file, respectively. (If **−−detach** is specified, the daemon closes its standard file descriptors, so logging to the console will have no effect.)
- **off**, **emer**, **err**, **warn**, **info**, or **dbg**, to control the log level. Messages of the given severity or higher will be logged, and messages of lower severity will be filtered out. **off** filters out all messages. See **ovs−appctl**(8) for a definition of each log level.

Case is not significant within *spec*.

Regardless of the log levels set for **file**, logging to a file will not take place unless **−−log−file** is also specified (see below).

For compatibility with older versions of OVS, **any** is accepted as a word but has no effect.

−v
−−verbose
Sets the maximum logging verbosity level, equivalent to **−−verbose=dbg**.

−vPATTERN:*destination***:***pattern*
−−verbose=PATTERN:*destination***:***pattern*
Sets the log pattern for *destination* to *pattern*. Refer to **ovs−appctl**(8) for a description of the valid syntax for *pattern*.

−vFACILITY:*facility*
−−verbose=FACILITY:*facility*
Sets the RFC5424 facility of the log message. *facility* can be one of **kern**, **user**, **mail**, **daemon**, **auth**, **syslog**, **lpr**, **news**, **uucp**, **clock**, **ftp**, **ntp**, **audit**, **alert**, **clock2**, **local0**, **local1**, **local2**, **local3**, **local4**, **local5**, **local6** or **local7**. If this option is not specified, **daemon** is used as the default for the local system syslog and **local0** is used while sending a message to the target provided via the **−−syslog−target** option.

−−log−file[**=***file*]
Enables logging to a file. If *file* is specified, then it is used as the exact name for the log file. The default log file name used if *file* is omitted is **/usr/local/var/log/ovn/***program***.log**.

−−syslog−target=*host***:***port*
Send syslog messages to UDP *port* on *host*, in addition to the system syslog. The *host* must be a numerical IP address, not a hostname.

−−syslog−method=*method*
Specify *method* as how syslog messages should be sent to syslog daemon. The following forms are supported:

- **libc**, to use the libc **syslog()** function. Downside of using this options is that libc adds fixed prefix to every message before it is actually sent to the syslog daemon over **/dev/log** UNIX domain socket.
- **unix:***file*, to use a UNIX domain socket directly. It is possible to specify arbitrary message format with this option. However, **rsyslogd 8.9** and older versions use hard coded parser function anyway that limits UNIX domain socket use. If you want to use arbitrary message format with older **rsyslogd** versions, then use UDP socket to localhost IP address instead.
- **udp:***ip***:***port*, to use a UDP socket. With this method it is possible to use arbitrary message format also with older **rsyslogd**. When sending syslog messages over UDP socket extra precaution needs to be taken into account, for example, syslog daemon needs to be configured to listen on the specified UDP port, accidental iptables rules could be interfering with local syslog traffic and there are some security considerations that apply to UDP sockets, but do not apply to UNIX domain sockets.
- **null**, to discard all messages logged to syslog.

The default is taken from the **OVS_SYSLOG_METHOD** environment variable; if it is unset, the default is **libc**.

Table Formatting Options
These options control the format of output from the **list** and **find** commands.

−f *format*
−−format=*format*
Sets the type of table formatting. The following types of *format* are available:

table 2-D text tables with aligned columns.

list (default)
A list with one column per line and rows separated by a blank line.

html HTML tables.

csv Comma-separated values as defined in RFC 4180.

json JSON format as defined in RFC 4627. The output is a sequence of JSON objects, each of which corresponds to one table. Each JSON object has the following members with the noted values:

caption
The table's caption. This member is omitted if the table has no caption.

headings
An array with one element per table column. Each array element is a string giving the corresponding column's heading.

data An array with one element per table row. Each element is also an array with one element per table column. The elements of this second-level array are the cells that constitute the table. Cells that represent OVSDB data or data types are expressed in the format described in the OVSDB specification; other cells are simply expressed as text strings.

−d *format*
−−data=*format*
Sets the formatting for cells within output tables unless the table format is set to **json**, in which case **json** formatting is always used when formatting cells. The following types of *format* are available:

string (default)
The simple format described in the **Database Values** section of **ovs−vsctl**(8).

bare The simple format with punctuation stripped off: **[]** and **{}** are omitted around sets, maps, and empty columns, items within sets and maps are space-separated, and strings are never quoted. This format may be easier for scripts to parse.

json The RFC 4627 JSON format as described above.

−−no−headings
This option suppresses the heading row that otherwise appears in the first row of table output.

−−pretty
By default, JSON in output is printed as compactly as possible. This option causes JSON in output to be printed in a more readable fashion. Members of objects and elements of arrays are printed one per line, with indentation.

This option does not affect JSON in tables, which is always printed compactly.

−−bare Equivalent to **−−format=list −−data=bare −−no−headings**.

PKI Options

PKI configuration is required to use SSL/TLS for the connection to the database.

−p *privkey.pem*
−−private−key=*privkey.pem*
Specifies a PEM file containing the private key used as identity for outgoing SSL/TLS connections.

−c *cert.pem*
−−certificate=*cert.pem*
Specifies a PEM file containing a certificate that certifies the private key specified on **−p** or **−−private−key** to be trustworthy. The certificate must be signed by the certificate authority (CA) that the peer in SSL/TLS connections will use to verify it.

−C *cacert.pem*
−−ca−cert=*cacert.pem*
Specifies a PEM file containing the CA certificate for verifying certificates presented to this program by SSL/TLS peers. (This may be the same certificate that SSL/TLS peers use to verify the certificate specified on **−c** or **−−certificate**, or it may be a different one, depending on the PKI design in use.)

−C none
−−ca−cert=none
Disables verification of certificates presented by SSL/TLS peers. This introduces a security risk, because it means that certificates cannot be verified to be those of known trusted hosts.

−−ssl−server−name=*servername*
Specifies the server name to use for TLS Server Name Indication (SNI). By default, the hostname from the connection string is used for SNI. This option allows overriding the SNI hostname, which is useful when connecting through proxies or service meshes where the connection endpoint differs from the intended server name.

−−bootstrap−ca−cert=*cacert.pem*
When *cacert.pem* exists, this option has the same effect as **−C** or **−−ca−cert**. If it does not exist, then the executable will attempt to obtain the CA certificate from the SSL/TLS peer on its first SSL/TLS connection and save it to the named PEM file. If it is successful, it will immediately drop the connection and reconnect, and from then on all SSL/TLS connections must be authenticated by a certificate signed by the CA certificate thus obtained.

This option exposes the SSL/TLS connection to a man-in-the-middle attack obtaining the initial CA certificate, but it may be useful for bootstrapping.

This option is only useful if the SSL/TLS peer sends its CA certificate as part of the SSL/TLS certificate chain. SSL/TLS protocols do not require the server to send the CA certificate.

This option is mutually exclusive with **−C** and **−−ca−cert**.

Other Options

−h
−−help Prints a brief help message to the console.

−V
−−version
Prints version information to the console.

COMMANDS

The following sections describe the commands that **ovn−sbctl** supports.

OVN_Southbound Commands

These commands work with an **OVN_Southbound** database as a whole.

init Initializes the database, if it is empty. If the database has already been initialized, this command has no effect.

[**−−filter=***filter-rule*[,*filter-rule*]...] **show**
Prints a brief overview of the database contents. If **−−filter** is specified, output is filtered according to the rules. Each *filter-rule* has the form *table-name*(*filter*[|*filter*]...). If *table-name* is omitted, filtering is applied to the **Chassis** table by default. A row is shown only if its printed representation (including all referenced rows) contains at least one of the specified *filter* substrings.

Chassis Commands

These commands manipulate **OVN_Southbound** chassis.

[**−−may−exist**] **chassis−add** *chassis encap-type encap-ip*
Creates a new chassis named *chassis*. *encap-type* is a comma-separated list of tunnel types. The chassis will have one encap entry for each specified tunnel type with *encap-ip* as the destination IP for each.

Without **−−may−exist**, attempting to create a chassis that exists is an error. With **−−may−exist**, this command does nothing if *chassis* already exists.

[**−−if−exists**] **chassis−del** *chassis*
Deletes *chassis*, its *encaps* and the associated Chassis_Private record (if it exists).

Without **−−if−exists**, attempting to delete a chassis that does not exist is an error. With **−−if−exists**, attempting to delete a chassis that does not exist has no effect.

Port Binding Commands

These commands manipulate **OVN_Southbound** port bindings.

[**−−may−exist**] **lsp−bind** *logical-port chassis*
Binds the logical port named *logical-port* to *chassis*.

Without **−−may−exist**, attempting to bind a logical port that has already been bound is an error. With **−−may−exist**, this command does nothing if *logical-port* has already been bound to the specified chassis. It is still an error if the port is bound to a different chassis.

[**−−if−exists**] **lsp−unbind** *logical-port*
Removes the binding of *logical-port*.

Without **−−if−exists**, attempting to unbind a logical port that is not bound is an error. With **−−if−exists**, attempting to unbind a logical port that is not bound has no effect.

Logical Flow Commands

[**−−uuid**] [**−−ovs**[**=***remote*]] [**−−stats**] [**−−vflows**] **lflow−list** [*logical-datapath*] [*lflow*...]

List logical flows. If *logical-datapath* is specified, only list flows for that logical datapath. The *logical-datapath* may be given as a UUID or as a datapath name (reporting an error if multiple datapaths have the same name).

If at least one *lflow* is given, only matching logical flows, if any, are listed. Each *lflow* may be specified as a UUID or the first few characters of a UUID, optionally prefixed by **0x**. (Because **ovn−controller** sets OpenFlow flow cookies to the first 32 bits of the corresponding logical flow's UUID, this makes it easy to look up the logical flow that generated a particular OpenFlow flow.)

If **−−uuid** is specified, the output includes the first 32 bits of each logical flow's UUID. This makes it easier to find the OpenFlow flows that correspond to a given logical flow.

If **−−ovs** is included, **ovn−sbctl** attempts to obtain and display the OpenFlow flows that correspond to each OVN logical flow. To do so, **ovn−sbctl** connects to *remote* (by default, **unix:/br−int.mgmt**) over OpenFlow and retrieves the flows. If *remote* is specified, it must be an active OpenFlow connection method described in **ovsdb**(7). Please see the discussion of the similar **−−ovs** option in **ovn−trace**(8) for more information about the OpenFlow flow output.

By default, OpenFlow flow output includes only match and actions. Add **−−stats** to include all OpenFlow information, such as packet and byte counters, duration, and timeouts.

If **−−vflows** is included, other southbound database records directly used for generating OpenFlow flows are also listed. This includes: *port-bindings*, *mac-bindings*, *multicast-groups*, *chassis* and *load-balancers*. The **−−ovs** and **−−stats** options can also be used in conjunction with **−−vflows**.

dump−flows

Alias for **lflow-list** and accepts all the same options.

count−flows [*logical-datapath*]

Prints the number of logical flows per table and per datapath.

IP Multicast Commands

ip−multicast−flush [*switch*]

Flushes the learned IP multicast groups for the logical switch identified by *switch*, which may be given as a UUID or as a datapath name. If *switch* is not specified, flushes learned multicast groups for all logical switches.

This works by incrementing the **seq_no** column of the **IP_Multicast** record for the datapath. **ovn−controller** monitors this value and flushes its locally learned multicast groups when it detects a change.

Remote Connectivity Commands

These commands manipulate the **connections** column in the **SB_Global** table and rows in the **Connection** table. When **ovsdb−server** is configured to use the **connections** column for OVSDB connections, this allows the administrator to use **ovn−sbctl** to configure database connections.

get−connection

Prints the configured connection(s).

del−connection

Deletes the configured connection(s).

[**−−inactivity−probe=***msecs*] **set−connection** *target*...

Sets the configured connection target or targets. Use **−−inactivity−probe=***msecs* to override the default idle connection inactivity probe time. Use 0 to disable inactivity probes.

Each *target* may be preceded by zero or more of the following modifiers, which apply to all subsequent targets until changed:

read−only
Sets the **read_only** column for subsequent connections to **true**, restricting them to read-only transactions.

read−write
Sets the **read_only** column for subsequent connections to **false**, allowing them to modify the database. This is the default.

role=*name*
Sets the **role** column for subsequent connections to *name*. The default is the empty string.

SSL/TLS Configuration Commands

When **ovsdb−server** is configured to connect using SSL/TLS, the following parameters are required:

private-key
Specifies a PEM file containing the private key used for SSL/TLS connections.

certificate
Specifies a PEM file containing a certificate, signed by the certificate authority (CA) used by the connection peers, that certifies the private key, identifying a trustworthy peer.

ca-cert Specifies a PEM file containing the CA certificate used to verify that the connection peers are trustworthy.

These SSL/TLS settings apply to all SSL/TLS connections made by the southbound database server.

get−ssl Prints the SSL/TLS configuration.

del−ssl Deletes the current SSL/TLS configuration.

[**−−bootstrap**] **set−ssl** *private-key certificate ca-cert* [*ssl-protocol-list* [*ssl-cipher-list* [*ssl-ciphersuites*]]]
Sets the SSL/TLS configuration.

Database Commands

These commands query and modify the contents of **ovsdb** tables. They are a slight abstraction of the **ovsdb** interface and as such they operate at a lower level than other **ovn−sbctl** commands.

Identifying Tables, Records, and Columns

Each of these commands has a *table* parameter to identify a table within the database. Many of them also take a *record* parameter that identifies a particular record within a table. The *record* parameter may be the UUID for a record, which may be abbreviated to its first 4 (or more) hex digits, as long as that is unique. Many tables offer additional ways to identify records. Some commands also take *column* parameters that identify a particular field within the records in a table.

For a list of tables and their columns, see **ovn−sb**(5) or see the table listing from the **−−help** option.

Record names must be specified in full and with correct capitalization, except that UUIDs may be abbreviated to their first 4 (or more) hex digits, as long as that is unique within the table. Names of tables and columns are not case-sensitive, and **−** and **_** are treated interchangeably. Unique abbreviations of table and column names are acceptable, e.g. **d** or **dhcp** is sufficient to identify the **DHCP_Options** table.

Database Values

Each column in the database accepts a fixed type of data. The currently defined basic types, and their representations, are:

integer A decimal integer in the range −2**63 to 2**63−1, inclusive.

real A floating-point number.

Boolean
True or false, written **true** or **false**, respectively.

string An arbitrary Unicode string, except that null bytes are not allowed. Quotes are optional for most strings that begin with an English letter or underscore and consist only of letters, underscores, hyphens, and periods. However, **true** and **false** and strings that match the syntax of UUIDs (see below) must be enclosed in double quotes to distinguish them from other basic types. When double quotes are used, the syntax is that of strings in JSON, e.g. backslashes may be used to escape special characters. The empty string must be represented as a pair of double quotes (**""**).

UUID Either a universally unique identifier in the style of RFC 4122, e.g. **f81d4fae−7dec−11d0−a765−00a0c91e6bf6**, or an @*name* defined by a **get** or **create** command within the same **ovs−vsctl** invocation.

Multiple values in a single column may be separated by spaces or a single comma. When multiple values are present, duplicates are not allowed, and order is not important. Conversely, some database columns can have an empty set of values, represented as **[]**, and square brackets may optionally enclose other non-empty sets or single values as well.

A few database columns are ‘‘maps’’ of key-value pairs, where the key and the value are each some fixed database type. These are specified in the form *key***=***value*, where *key* and *value* follow the syntax for the column’s key type and value type, respectively. When multiple pairs are present (separated by spaces or a comma), duplicate keys are not allowed, and again the order is not important. Duplicate values are allowed. An empty map is represented as **{}**. Curly braces may optionally enclose non-empty maps as well (but use quotes to prevent the shell from expanding **other−config={0=x,1=y}** into **other−config=0=x other−config=1=y**, which may not have the desired effect).

Database Command Syntax

[**−−if−exists**] [**−−columns=***column*[**,***column*]...] **list** *table* [*record*]...

Lists the data in each specified *record*. If no records are specified, lists all the records in *table*.

If **−−columns** is specified, only the requested columns are listed, in the specified order. Otherwise, all columns are listed, in alphabetical order by column name.

Without **−−if−exists**, it is an error if any specified *record* does not exist. With **−−if−exists**, the command ignores any *record* that does not exist, without producing any output.

[**−−columns=***column*[**,***column*]...] **find** *table* [*column*[**:***key*]**=***value*]...

Lists the data in each record in *table* whose *column* equals *value* or, if *key* is specified, whose *column* contains a *key* with the specified *value*. The following operators may be used where **=** is written in the syntax summary:

= != < > <= >=

Selects records in which *column*[**:***key*] equals, does not equal, is less than, is greater than, is less than or equal to, or is greater than or equal to *value*, respectively.

Consider *column*[**:***key*] and *value* as sets of elements. Identical sets are considered equal. Otherwise, if the sets have different numbers of elements, then the set with more elements is considered to be larger. Otherwise, consider a element from each set pairwise, in increasing order within each set. The first pair that differs determines the result. (For a column that contains key-value pairs, first all the keys are compared, and values are considered only if the two sets contain identical keys.)

{=} {!=}

Test for set equality or inequality, respectively.

{<=} Selects records in which *column*[**:***key*] is a subset of *value*. For example, **flood−vlans{<=}1,2** selects records in which the **flood−vlans** column is the empty set or contains 1 or 2 or both.

{<} Selects records in which *column*[**:***key*] is a proper subset of *value*. For example, **flood−vlans{<}1,2** selects records in which the **flood−vlans** column is the empty set or contains 1 or 2 but not both.

{>=} {>}
Same as **{<=}** and **{<}**, respectively, except that the relationship is reversed. For example, **flood−vlans{>=}1,2** selects records in which the **flood−vlans** column contains both 1 and 2.

The following operators are available only in Open vSwitch 2.16 and later:

{in} Selects records in which every element in *column*[**:***key*] is also in *value*. (This is the same as **{<=}**.)

{not−in}
Selects records in which every element in *column*[**:***key*] is not in *value*.

For arithmetic operators (**= != < > <= >=**), when *key* is specified but a particular record's *column* does not contain *key*, the record is always omitted from the results. Thus, the condition **other−config:mtu!=1500** matches records that have a **mtu** key whose value is not 1500, but not those that lack an **mtu** key.

For the set operators, when *key* is specified but a particular record's *column* does not contain *key*, the comparison is done against an empty set. Thus, the condition **other−config:mtu{!=}1500** matches records that have a **mtu** key whose value is not 1500 and those that lack an **mtu** key.

Don't forget to escape **<** or **>** from interpretation by the shell.

If **−−columns** is specified, only the requested columns are listed, in the specified order. Otherwise all columns are listed, in alphabetical order by column name.

The UUIDs shown for rows created in the same **ovs−vsctl** invocation will be wrong.

[**−−if−exists**] [**−−id=@***name*] **get** *table record* [*column*[**:***key*]]...
Prints the value of each specified *column* in the given *record* in *table*. For map columns, a *key* may optionally be specified, in which case the value associated with *key* in the column is printed, instead of the entire map.

Without **−−if−exists**, it is an error if *record* does not exist or *key* is specified, if *key* does not exist in *record*. With **−−if−exists**, a missing *record* yields no output and a missing *key* prints a blank line.

If **@***name* is specified, then the UUID for *record* may be referred to by that name later in the same **ovs−vsctl** invocation in contexts where a UUID is expected.

Both **−−id** and the *column* arguments are optional, but usually at least one or the other should be specified. If both are omitted, then **get** has no effect except to verify that *record* exists in *table*.

−−id and **−−if−exists** cannot be used together.

[**−−if−exists**] **set** *table record column*[**:***key*]**=***value*...
Sets the value of each specified *column* in the given *record* in *table* to *value*. For map columns, a *key* may optionally be specified, in which case the value associated with *key* in that column is changed (or added, if none exists), instead of the entire map.

Without **−−if−exists**, it is an error if *record* does not exist. With **−−if−exists**, this command does nothing if *record* does not exist.

[**−−if−exists**] **add** *table record column* [*key***=**]*value*...
Adds the specified value or key-value pair to *column* in *record* in *table*. If *column* is a map, then *key* is required, otherwise it is prohibited. If *key* already exists in a map column, then the current *value* is not replaced (use the **set** command to replace an existing value).

Without **−−if−exists**, it is an error if *record* does not exist. With **−−if−exists**, this command does nothing if *record* does not exist.

[**−−if−exists**] **remove** *table record column value...*

[**−−if−exists**] **remove** *table record column key...*

[**−−if−exists**] **remove** *table record column key*=*value...* Removes the specified values or key-value pairs from *column* in *record* in *table*. The first form applies to columns that are not maps: each specified *value* is removed from the column. The second and third forms apply to map columns: if only a *key* is specified, then any key-value pair with the given *key* is removed, regardless of its value; if a *value* is given then a pair is removed only if both key and value match.

It is not an error if the column does not contain the specified key or value or pair.

Without **−−if−exists**, it is an error if *record* does not exist. With **−−if−exists**, this command does nothing if *record* does not exist.

[**−−if−exists**] **clear** *table record column...*

Sets each *column* in *record* in *table* to the empty set or empty map, as appropriate. This command applies only to columns that are allowed to be empty.

Without **−−if−exists**, it is an error if *record* does not exist. With **−−if−exists**, this command does nothing if *record* does not exist.

[**−−id=**(@*name*|*uuid*)] **create** *table column*[**:***key*]=*value...*

Creates a new record in *table* and sets the initial values of each *column*. Columns not explicitly set will receive their default values. Outputs the UUID of the new row.

If @*name* is specified, then the UUID for the new row may be referred to by that name elsewhere in the same ***(PN** invocation in contexts where a UUID is expected. Such references may precede or follow the **create** command.

If a valid *uuid* is specified, then it is used as the UUID of the new row.

Caution (ovs-vsctl as example)

Records in the Open vSwitch database are significant only when they can be reached directly or indirectly from the **Open_vSwitch** table. Except for records in the **QoS** or **Queue** tables, records that are not reachable from the **Open_vSwitch** table are automatically deleted from the database. This deletion happens immediately, without waiting for additional **ovs−vsctl** commands or other database activity. Thus, a **create** command must generally be accompanied by additional commands *within the same* **ovs−vsctl** *invocation* to add a chain of references to the newly created record from the top-level **Open_vSwitch** record. The **EXAMPLES** section gives some examples that show how to do this.

[**−−if−exists**] **destroy** *table record...*

Deletes each specified *record* from *table*. Unless **−−if−exists** is specified, each *record*s must exist.

−−all destroy *table*

Deletes all records from the *table*.

Caution (ovs-vsctl as example)

The **destroy** command is only useful for records in the **QoS** or **Queue** tables. Records in other tables are automatically deleted from the database when they become unreachable from the **Open_vSwitch** table. This means that deleting the last reference to a record is sufficient for deleting the record itself. For records in these tables, **destroy** is silently ignored. See the **EXAMPLES** section below for more information.

wait−until *table record* [*column*[**:***key*]**=***value*]...
Waits until *table* contains a record named *record* whose *column* equals *value* or, if *key* is specified, whose *column* contains a *key* with the specified *value*. This command supports the same operators and semantics described for the **find** command above.

If no *column*[**:***key*]**=***value* arguments are given, this command waits only until *record* exists. If more than one such argument is given, the command waits until all of them are satisfied.

Caution (ovs-vsctl as example)
Usually **wait−until** should be placed at the beginning of a set of **ovs−vsctl** commands. For example, **wait−until bridge br0 −− get bridge br0 datapath_id** waits until a bridge named **br0** is created, then prints its **datapath_id** column, whereas **get bridge br0 datapath_id −− wait−until bridge br0** will abort if no bridge named **br0** exists when **ovs−vsctl** initially connects to the database.

Consider specifying **−−timeout=0** along with **−−wait−until**, to prevent **ovs−vsctl** from terminating after waiting only at most 5 seconds.

comment [*arg*]...
This command has no effect on behavior, but any database log record created by the command will include the command and its arguments.

ENVIRONMENT

OVN_SB_DAEMON
If set, this should name the Unix domain socket for an **ovn−sbctl** server process. See **Daemon Mode**, above, for more information.

OVN_SBCTL_OPTIONS
If set, a set of options for **ovn−sbctl** to apply automatically, in the same form as on the command line.

OVN_SB_DB
If set, the default database to contact when the **−−db** option is not used.

EXIT STATUS

0 Successful program execution.

1 Usage, syntax, or network error.

SEE ALSO

ovn−sb(5), **ovn−appctl**(8).